

In []:

Signals and Systems

Module 4

Fourier Transform of Continuous-Time Signals

Introduction

The Fourier Transform is one of the most important mathematical tools used in signal processing. It converts a time-domain signal into its frequency-domain representation, allowing engineers to analyze the frequency components present in a signal. This notebook demonstrates the computation, visualization, and properties of Fourier Transforms using Python.

4.1 Definition and Inverse Fourier Transform

Aim

- To understand the Fourier Transform.
- To visualize the frequency spectrum of signals.
- To reconstruct the original signal using the Inverse Fourier Transform.

Problem 1

Problem Statement

Generate the continuous-time signal

$$x(t) = \sin(2\pi 5t)$$

Compute its Fourier Transform using the Fast Fourier Transform (FFT) and plot:

- Time-domain signal

- Magnitude spectrum

```
In [15]: import numpy as np
import matplotlib.pyplot as plt

# Sampling parameters
fs = 100
t = np.arange(0,1,1/fs)

# Signal
x = np.sin(2*np.pi*5*t)

# Fourier Transform
X = np.fft.fft(x)
freq = np.fft.fftfreq(len(x),1/fs)

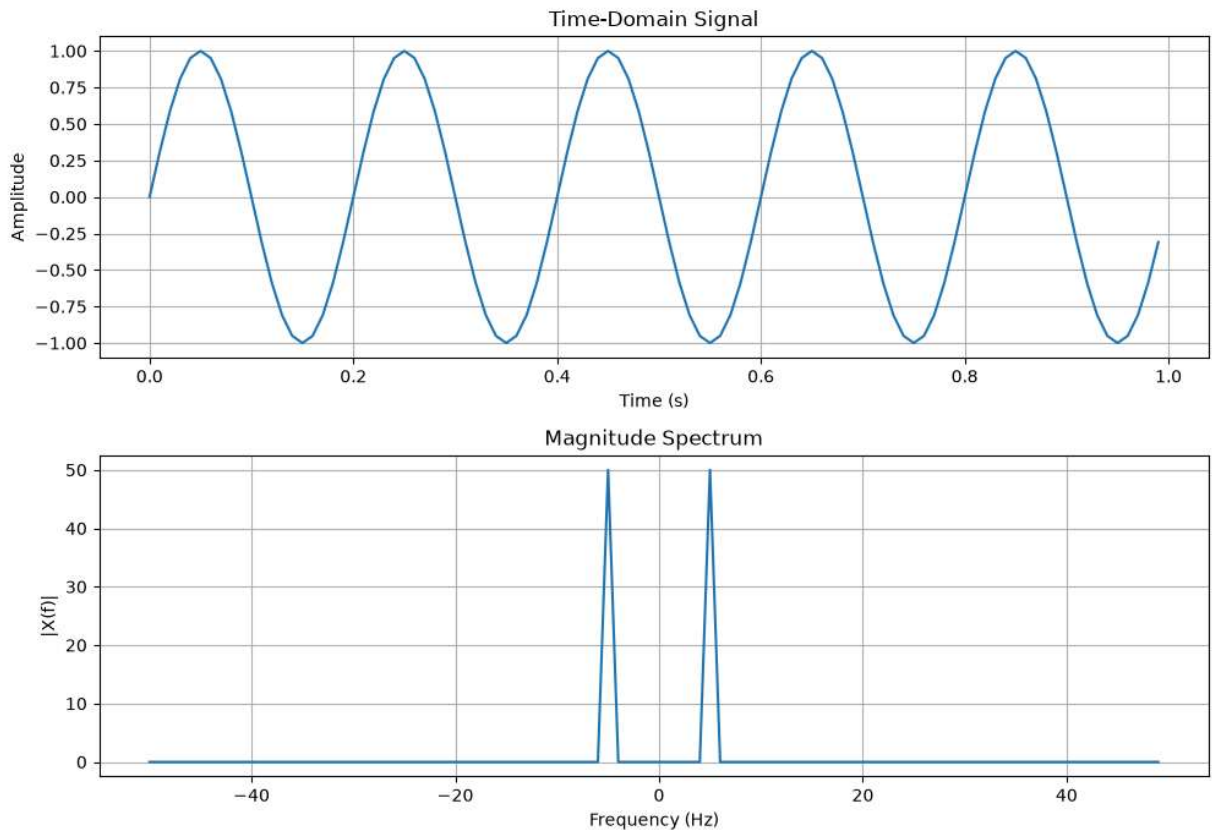
plt.figure(figsize=(10,7))

plt.subplot(2,1,1)
plt.plot(t,x)
plt.title("Time-Domain Signal")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(2,1,2)
plt.plot(np.fft.fftshift(freq),
         np.fft.fftshift(np.abs(X)))

plt.title("Magnitude Spectrum")
plt.xlabel("Frequency (Hz)")
plt.ylabel(" $|X(f)|$ ")
plt.grid(True)

plt.tight_layout()
plt.show()
```



Observation

- The Fourier Transform converts the signal from the time domain to the frequency domain.
- The spectrum contains peaks at ± 5 Hz, corresponding to the sinusoidal frequency.
- The FFT efficiently computes the discrete approximation of the Fourier Transform.

Problem 2

Problem Statement

Using the Fourier Transform obtained in Problem 1, reconstruct the original signal using the Inverse Fourier Transform (IFFT) and compare it with the original signal.

```
In [16]: import numpy as np
import matplotlib.pyplot as plt

fs = 100
t = np.arange(0,1,1/fs)

x = np.sin(2*np.pi*5*t)

X = np.fft.fft(x)

x_rec = np.fft.ifft(X)
```

```
plt.figure(figsize=(10,4))

plt.plot(t,x,label="Original Signal")

plt.plot(t,x_rec.real,'--',
         label="Reconstructed Signal")

plt.title("Inverse Fourier Transform")

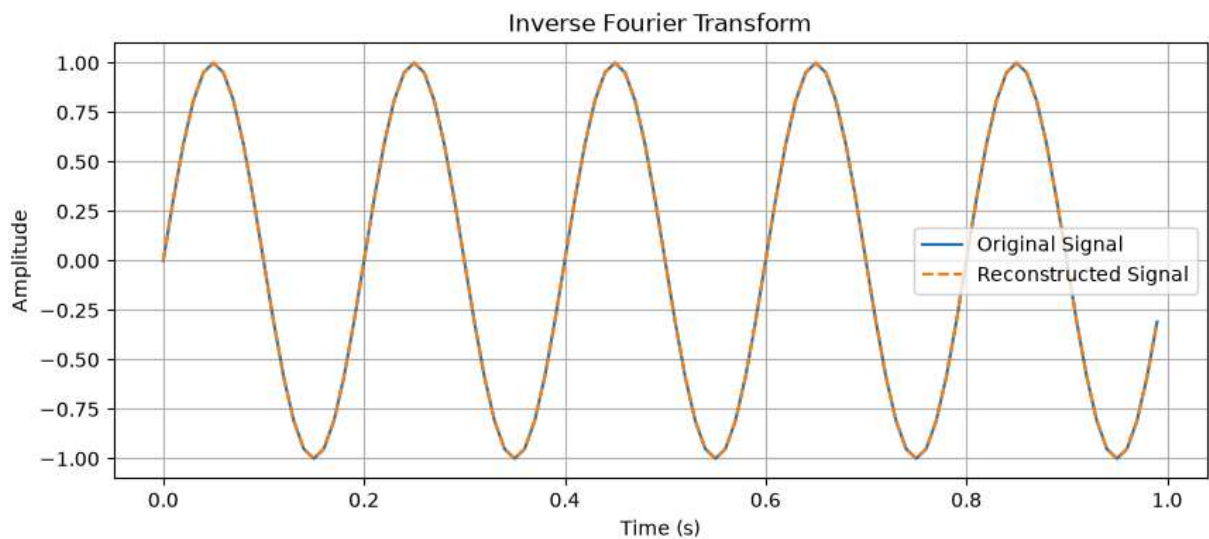
plt.xlabel("Time (s)")

plt.ylabel("Amplitude")

plt.legend()

plt.grid(True)

plt.show()
```



Observation

- The reconstructed signal overlaps almost exactly with the original signal.
- The Inverse Fourier Transform successfully recovers the time-domain signal.
- Numerical errors are negligible.

4.2 Conditions for Existence of Fourier Transform

Aim

- To study the conditions under which the Fourier Transform exists.
- To compare signals satisfying and violating the Fourier Transform existence conditions.

Problem 3

Problem Statement

Generate the exponentially decaying signal

$$x(t) = e^{-t}u(t)$$

Plot the signal and explain why its Fourier Transform exists.

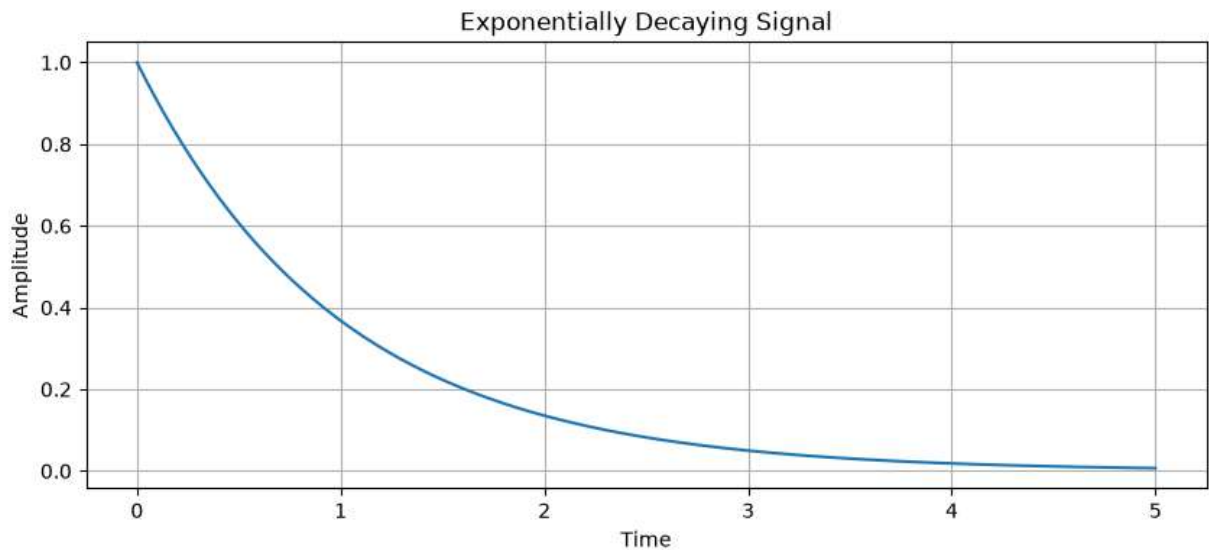
```
In [17]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,5,1000)
x = np.exp(-t)

plt.figure(figsize=(10,4))
plt.plot(t,x)

plt.title("Exponentially Decaying Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()

print("The signal is absolutely integrable.")
print("Hence its Fourier Transform exists.")
```



The signal is absolutely integrable.
Hence its Fourier Transform exists.

Observation

- The signal decays rapidly with time.
- The area under the signal is finite.
- Therefore, the Fourier Transform exists.

Problem 4

Problem Statement

Generate the exponentially growing signal

$$x(t) = e^t$$

over a finite interval and discuss whether its Fourier Transform exists over an infinite duration.

```
In [18]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,3,1000)

x = np.exp(t)

plt.figure(figsize=(10,4))

plt.plot(t,x)

plt.title("Exponentially Growing Signal")

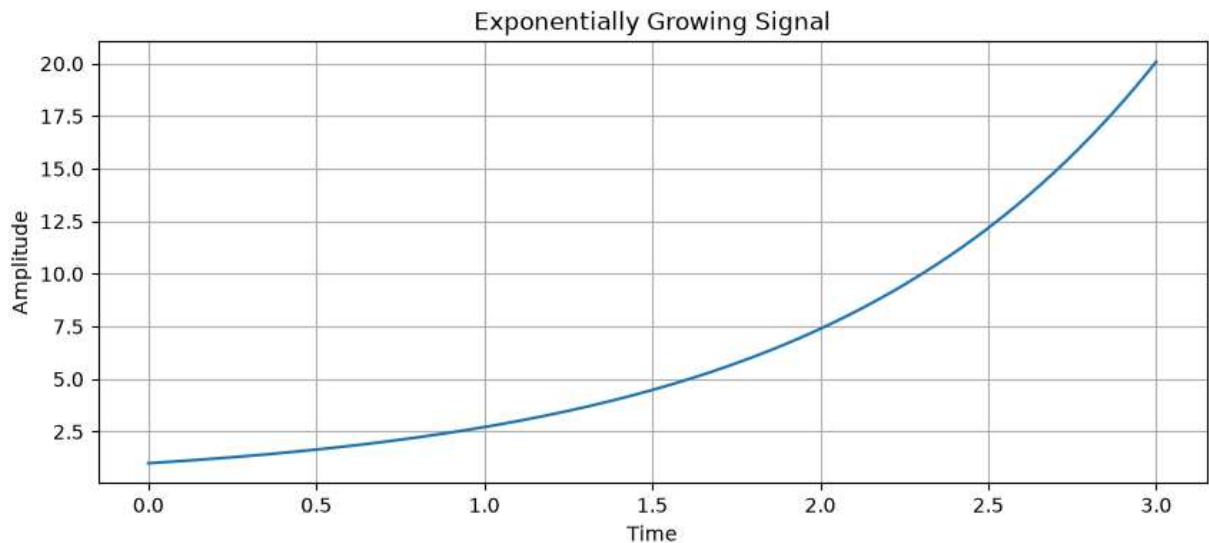
plt.xlabel("Time")
```

```
plt.ylabel("Amplitude")

plt.grid(True)

plt.show()

print("The signal grows without bound.")
print("Hence it is not absolutely integrable.")
print("Its Fourier Transform does not exist over an infinite interval.")
```



The signal grows without bound.
Hence it is not absolutely integrable.
Its Fourier Transform does not exist over an infinite interval.

Observation

- The signal increases exponentially with time.
- Its area becomes infinite over an infinite interval.
- Therefore, the Fourier Transform does not exist in the conventional sense.

4.3 Properties of Fourier Transform

Aim

- To study the important properties of the Fourier Transform.
- To verify these properties using numerical computation.
- To visualize the effect of different signal operations on the spectrum.

Problem 5

Problem Statement

Generate the signal

$$x(t) = \sin(2\pi 5t)$$

and verify the **linearity property** by comparing the Fourier Transform of

$$y(t) = 2x(t)$$

with the Fourier Transform of the original signal.

```
In [19]: import numpy as np
import matplotlib.pyplot as plt

fs = 100
t = np.arange(0,1,1/fs)

x = np.sin(2*np.pi*5*t)
y = 2*x

X = np.fft.fftshift(np.fft.fft(x))
Y = np.fft.fftshift(np.fft.fft(y))
freq = np.fft.fftshift(np.fft.fftfreq(len(t),1/fs))

plt.figure(figsize=(10,5))

plt.plot(freq,np.abs(X),label="|X(f)|")

plt.plot(freq,np.abs(Y),'--',label="|Y(f)|")

plt.title("Verification of Linearity Property")

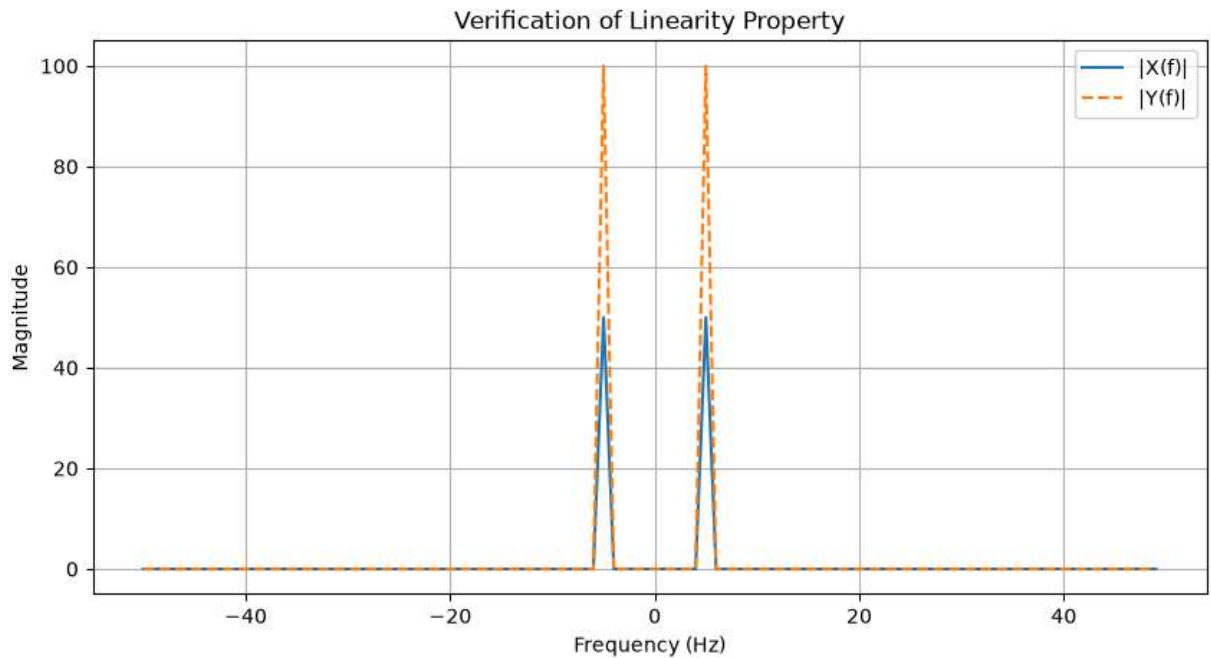
plt.xlabel("Frequency (Hz)")

plt.ylabel("Magnitude")

plt.grid(True)

plt.legend()

plt.show()
```



Observation

- Scaling the signal in the time domain scales its Fourier Transform by the same factor.
- The frequency locations remain unchanged.
- Hence, the linearity property is verified.

Problem 6

Problem Statement

Verify the **time-shifting property** of the Fourier Transform.

Generate

$$x(t) = \sin(2\pi 5t)$$

and its delayed version

$$x(t - 0.2)$$

Plot their magnitude spectra.

```
In [20]: import numpy as np
import matplotlib.pyplot as plt

fs = 100
t = np.arange(0, 1, 1/fs)

x = np.sin(2*np.pi*5*t)

delay = np.sin(2*np.pi*5*(t-0.2))
```

```
X = np.fft.fftshift(np.fft.fft(x))
D = np.fft.fftshift(np.fft.fft(delay))

freq = np.fft.fftshift(np.fft.fftfreq(len(t),1/fs))

plt.figure(figsize=(10,5))

plt.plot(freq,np.abs(X),label="Original")

plt.plot(freq,np.abs(D),'--',label="Delayed")

plt.title("Time Shifting Property")

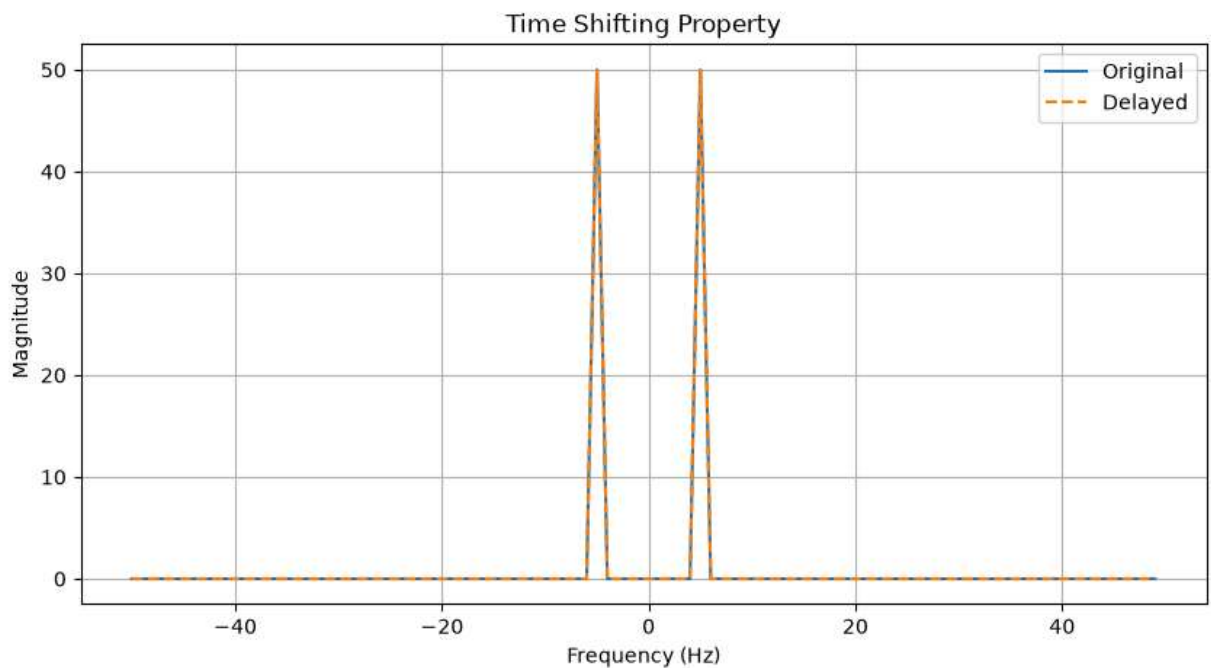
plt.xlabel("Frequency (Hz)")

plt.ylabel("Magnitude")

plt.grid(True)

plt.legend()

plt.show()
```



Observation

- Time shifting does not change the magnitude spectrum.
- Only the phase spectrum changes.
- Hence, the magnitude plots overlap.

4.3.1 Linearity, Time Shifting and Frequency Shifting

Aim

- To verify important Fourier Transform properties using FFT.

Problem 7

Problem Statement

Verify the **frequency shifting property** by multiplying

$$x(t) = \sin(2\pi 5t)$$

with

$$\cos(2\pi 20t)$$

Observe the shifted spectrum.

```
In [21]: import numpy as np
import matplotlib.pyplot as plt

fs=200

t=np.arange(0,1,1/fs)

x=np.sin(2*np.pi*5*t)

carrier=np.cos(2*np.pi*20*t)

y=x*carrier

Y=np.fft.fftshift(np.fft.fft(y))

freq=np.fft.fftshift(np.fft.fftfreq(len(t),1/fs))

plt.figure(figsize=(10,5))

plt.plot(freq,np.abs(Y))

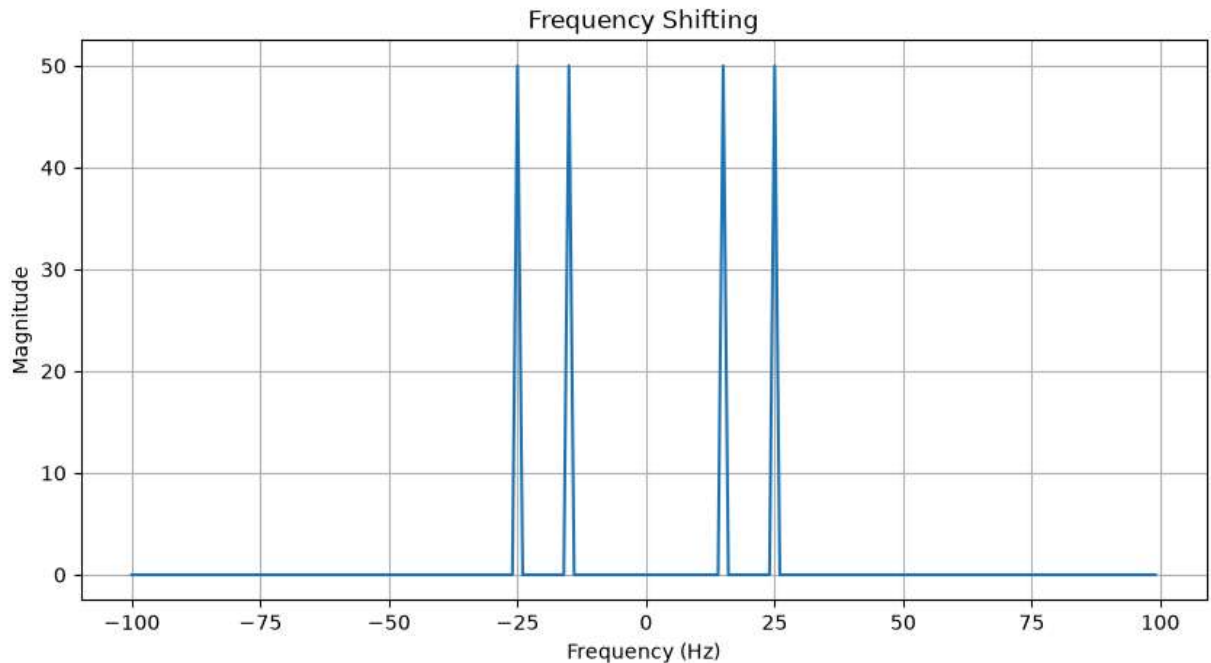
plt.title("Frequency Shifting")

plt.xlabel("Frequency (Hz)")

plt.ylabel("Magnitude")

plt.grid(True)
```

```
plt.show()
```



Observation

- Multiplication by a cosine shifts the spectrum.
- New frequency components appear around ± 25 Hz and ± 15 Hz.
- This demonstrates the modulation property.

Problem 8

Problem Statement

Compare the Fourier magnitude spectrum of a low-frequency sinusoid

$$\sin(2\pi 5t)$$

and a high-frequency sinusoid

$$\sin(2\pi 20t)$$

on the same graph.

```
In [22]: import numpy as np
import matplotlib.pyplot as plt

fs=200

t=np.arange(0,1,1/fs)

x1=np.sin(2*np.pi*5*t)
```

```

x2=np.sin(2*np.pi*20*t)

X1=np.fft.fftshift(np.fft.fft(x1))

X2=np.fft.fftshift(np.fft.fft(x2))

freq=np.fft.fftshift(np.fft.fftfreq(len(t),1/fs))

plt.figure(figsize=(10,5))

plt.plot(freq,np.abs(X1),label="5 Hz")

plt.plot(freq,np.abs(X2),label="20 Hz")

plt.title("Comparison of Frequency Spectra")

plt.xlabel("Frequency (Hz)")

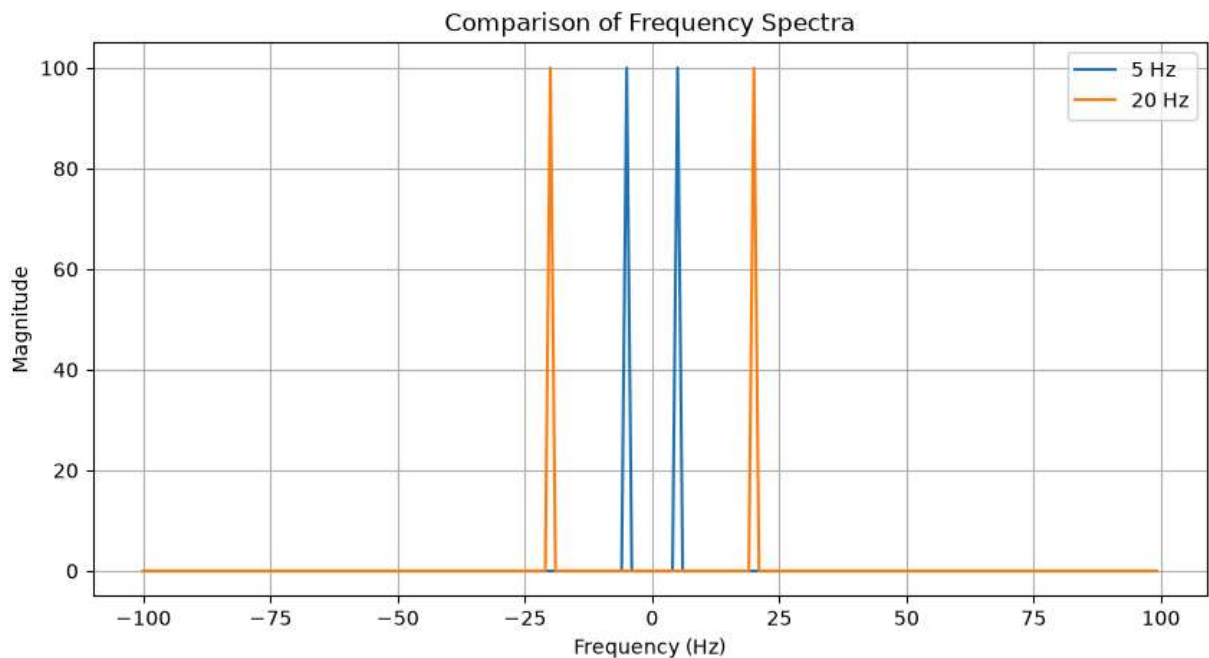
plt.ylabel("Magnitude")

plt.grid(True)

plt.legend()

plt.show()

```



Observation

- The higher-frequency signal produces spectral peaks farther from the origin.
- The FFT correctly identifies the dominant frequencies.

Problem 9

Problem Statement

Generate the signal

$$x(t) = \sin(2\pi 5t) + 0.5 \sin(2\pi 15t)$$

Compute and plot its Fourier magnitude spectrum.

```
In [23]: import numpy as np
import matplotlib.pyplot as plt

fs=200

t=np.arange(0,1,1/fs)

x=np.sin(2*np.pi*5*t)+0.5*np.sin(2*np.pi*15*t)

X=np.fft.fftshift(np.fft.fft(x))

freq=np.fft.fftshift(np.fft.fftfreq(len(t),1/fs))

plt.figure(figsize=(10,5))

plt.plot(freq,np.abs(X))

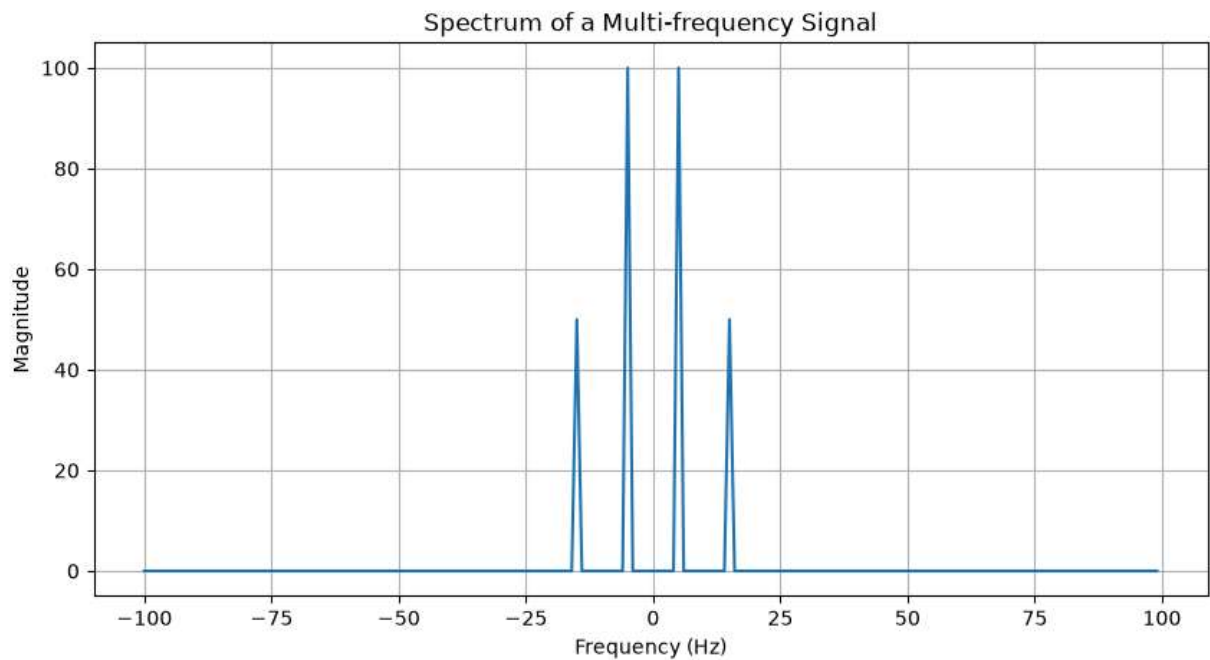
plt.title("Spectrum of a Multi-frequency Signal")

plt.xlabel("Frequency (Hz)")

plt.ylabel("Magnitude")

plt.grid(True)

plt.show()
```



Observation

- Two distinct frequency components are visible in the spectrum.
- The 5 Hz component has a larger magnitude than the 15 Hz component.
- The Fourier Transform separates the frequency components effectively.

Conclusion

The important properties of the Fourier Transform were verified using numerical FFT analysis. Linearity, time shifting, and frequency shifting were demonstrated graphically. The Fourier Transform was also used to identify the individual frequency components of composite signals, highlighting its significance in frequency-domain analysis.

4.4 Fourier Transform of Standard Signals

Aim

- To study the Fourier Transform of commonly used signals.
- To observe the relationship between time-domain and frequency-domain representations.
- To compare the spectra of different standard signals.

Problem 10

Problem Statement

Generate a rectangular pulse of width 0.4 seconds centered at the origin.

Compute its Fourier Transform using FFT and plot:

- Time-domain signal
- Magnitude spectrum

```
In [24]: import numpy as np
import matplotlib.pyplot as plt

fs = 200
t = np.linspace(-1,1,400)

x = np.where(np.abs(t)<=0.2,1,0)

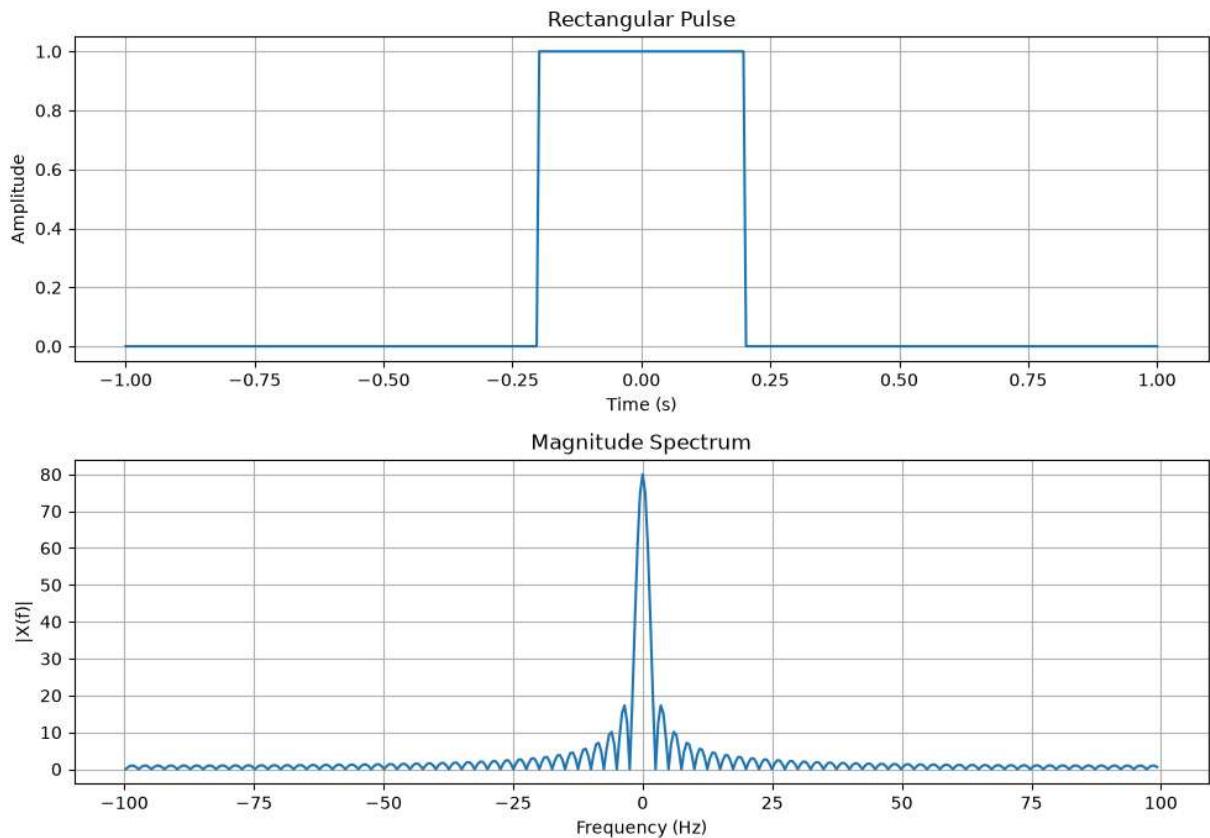
X = np.fft.fftshift(np.fft.fft(x))
freq = np.fft.fftshift(np.fft.fftfreq(len(t),d=t[1]-t[0]))

plt.figure(figsize=(10,7))

plt.subplot(2,1,1)
plt.plot(t,x)
plt.title("Rectangular Pulse")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(2,1,2)
plt.plot(freq,np.abs(X))
plt.title("Magnitude Spectrum")
plt.xlabel("Frequency (Hz)")
plt.ylabel("|X(f)|")
plt.grid(True)

plt.tight_layout()
plt.show()
```



Observation

- The rectangular pulse has a finite duration.
- Its Fourier Transform exhibits a sinc-like spectrum.
- A narrower pulse produces a wider frequency spectrum.

Problem 11

Problem Statement

Generate a Gaussian signal

$$x(t) = e^{-t^2}$$

Compute its Fourier Transform and plot both the signal and its magnitude spectrum.

```
In [25]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-5,5,1000)

x = np.exp(-t**2)

X = np.fft.fftshift(np.fft.fft(x))
freq = np.fft.fftshift(np.fft.fftfreq(len(t),d=t[1]-t[0]))
```

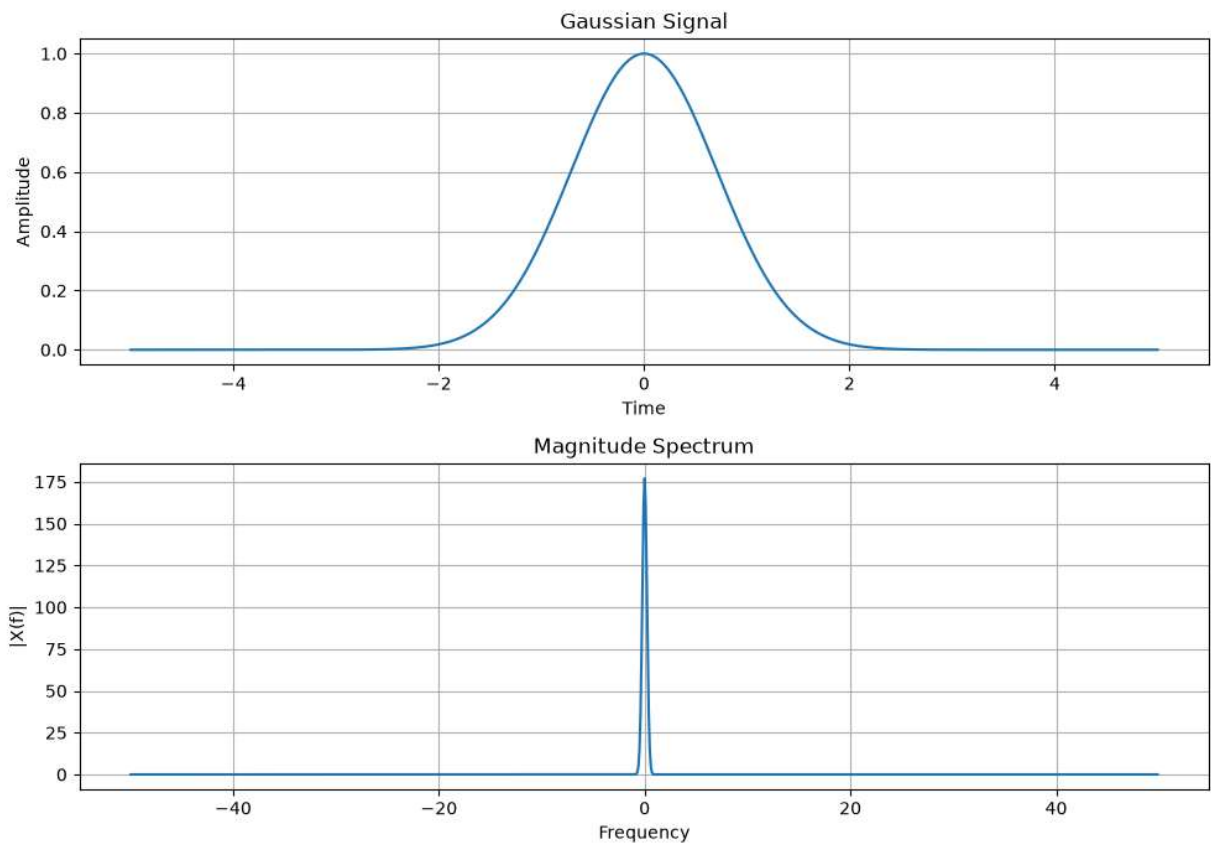
```
plt.figure(figsize=(10,7))

plt.subplot(2,1,1)
plt.plot(t,x)
plt.title("Gaussian Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(2,1,2)
plt.plot(freq,np.abs(X))
plt.title("Magnitude Spectrum")
plt.xlabel("Frequency")
plt.ylabel("|X(f)|")
plt.grid(True)

plt.tight_layout()

plt.show()
```



Observation

- The Gaussian signal is smooth and symmetric.
- Its Fourier Transform is also Gaussian.
- The Gaussian is one of the few signals whose Fourier Transform has the same general shape.

Problem 12

Problem Statement

Generate the exponentially decaying signal

$$x(t) = e^{-t}u(t)$$

Compute its Fourier Transform and plot its magnitude spectrum.

```
In [26]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,5,1000)

x = np.exp(-t)

X = np.fft.fftshift(np.fft.fft(x))
freq = np.fft.fftshift(np.fft.fftfreq(len(t),d=t[1]-t[0]))

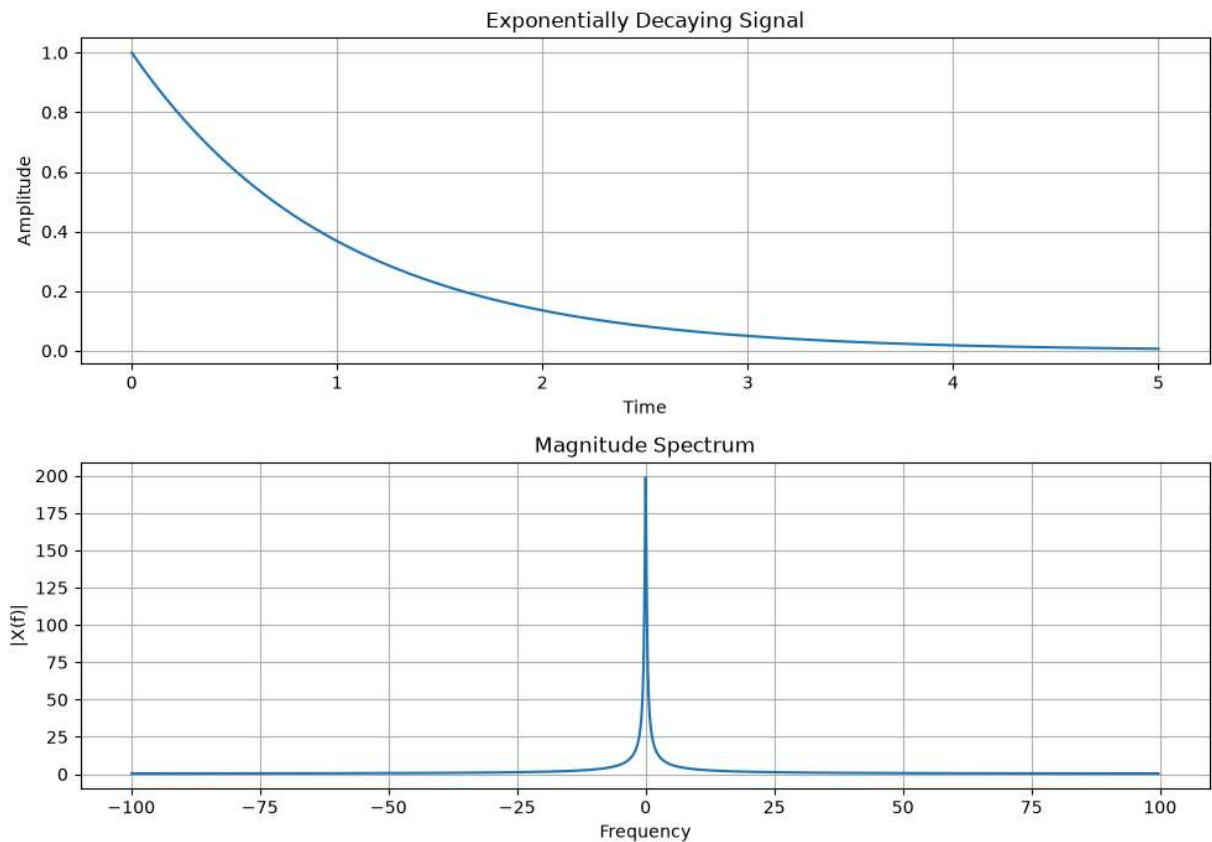
plt.figure(figsize=(10,7))

plt.subplot(2,1,1)
plt.plot(t,x)
plt.title("Exponentially Decaying Signal")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(2,1,2)
plt.plot(freq,np.abs(X))
plt.title("Magnitude Spectrum")
plt.xlabel("Frequency")
plt.ylabel("|X(f)|")
plt.grid(True)

plt.tight_layout()

plt.show()
```



Observation

- The exponentially decaying signal has finite energy.
- Its Fourier Transform is continuous.
- Most of the signal energy is concentrated around low frequencies.

Conclusion

In this section, the Fourier Transforms of standard signals such as the rectangular pulse, Gaussian signal, and exponentially decaying signal were studied. Their frequency-domain characteristics were analyzed using FFT. The experiments illustrate how the time-domain shape of a signal influences its frequency spectrum.

4.5 Parseval's Theorem and Energy Spectral Density

Aim

- To verify Parseval's Theorem numerically.
- To compute the Energy Spectral Density (ESD) of a signal.
- To compare the signal energy in both the time and frequency domains.

Problem 13

Problem Statement

Consider the signal

$$x(t) = \sin(2\pi 5t)$$

Verify **Parseval's Theorem** numerically by calculating the signal energy in:

- Time Domain
- Frequency Domain

Compare both values.

```
In [27]: import numpy as np
import matplotlib.pyplot as plt

# Sampling parameters
fs = 200
t = np.arange(0,1,1/fs)

# Signal
x = np.sin(2*np.pi*5*t)

# FFT
X = np.fft.fft(x)

# Time-domain Energy
E_time = np.sum(np.abs(x)**2)

# Frequency-domain Energy
E_freq = np.sum(np.abs(X)**2)/len(X)

print("Time Domain Energy =", E_time)
print("Frequency Domain Energy =", E_freq)

plt.figure(figsize=(10,4))
```

```
plt.plot(t,x)

plt.title("Signal used for Parseval Verification")

plt.xlabel("Time (s)")

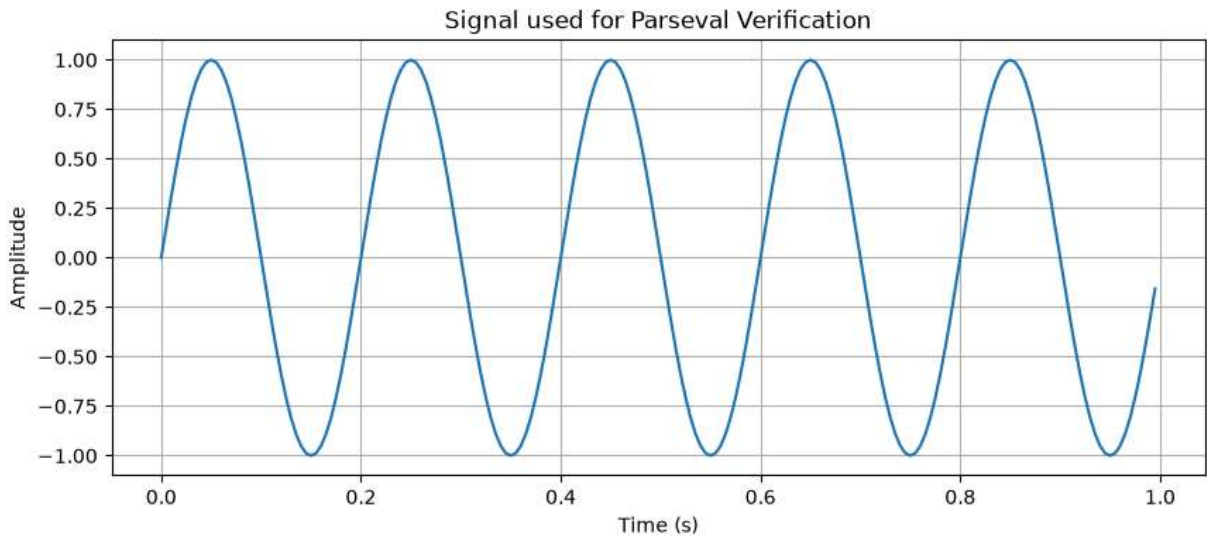
plt.ylabel("Amplitude")

plt.grid(True)

plt.show()
```

Time Domain Energy = 100.00000000000003

Frequency Domain Energy = 100.0



Observation

- The time-domain and frequency-domain energies are nearly identical.
- This verifies Parseval's Theorem numerically.
- Small differences may occur due to numerical approximation.

Problem 14

Problem Statement

Compute the **Energy Spectral Density (ESD)** of the signal

$$x(t) = \sin(2\pi 5t)$$

using the Fourier Transform and plot the Energy Spectral Density.

```
In [28]: import numpy as np
import matplotlib.pyplot as plt

fs = 200
```

```

t = np.arange(0,1,1/fs)

x = np.sin(2*np.pi*5*t)

X = np.fft.fftshift(np.fft.fft(x))

freq = np.fft.fftshift(np.fft.fftfreq(len(t),1/fs))

ESD = np.abs(X)**2

plt.figure(figsize=(10,5))

plt.plot(freq,ESD)

plt.title("Energy Spectral Density")

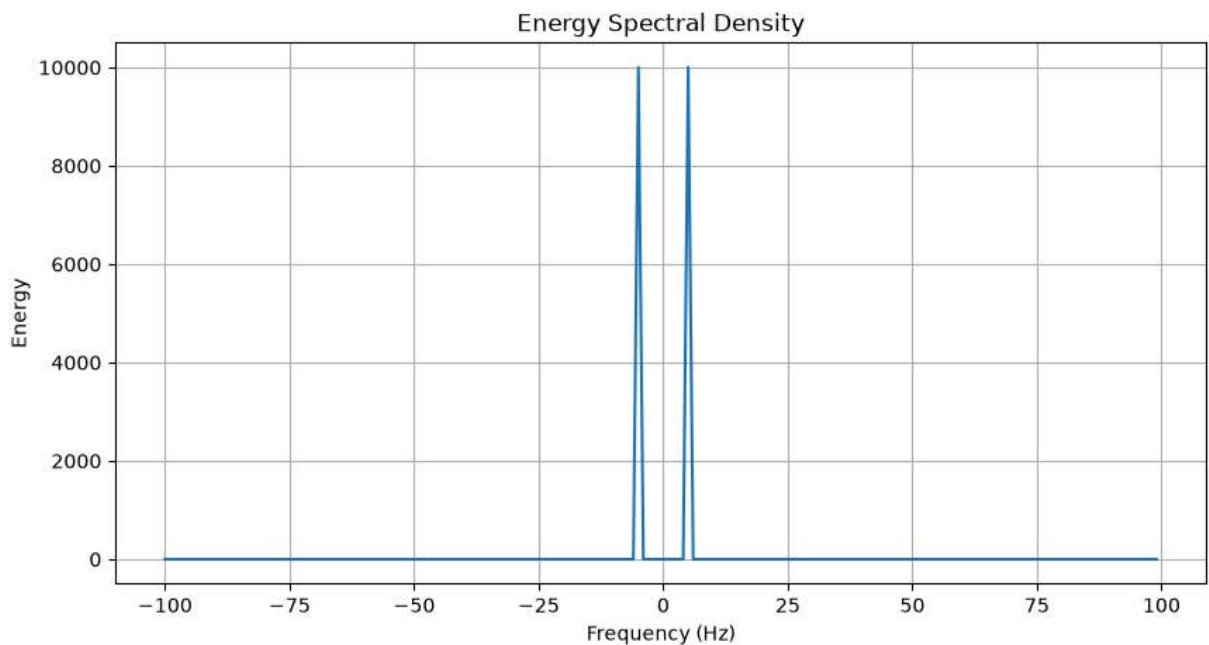
plt.xlabel("Frequency (Hz)")

plt.ylabel("Energy")

plt.grid(True)

plt.show()

```



Observation

- The Energy Spectral Density represents how the signal energy is distributed over frequency.
- The maximum energy is concentrated around ± 5 Hz.
- The ESD provides valuable information for frequency-domain signal analysis.

Module Conclusion

In this module, the Fourier Transform of continuous-time signals was studied using numerical computation. The Fast Fourier Transform (FFT) and Inverse Fast Fourier Transform (IFFT) were used to analyze signals in both the time and frequency domains. Important properties such as linearity, time shifting, and frequency shifting were verified. The Fourier Transforms of standard signals including rectangular, Gaussian, and exponentially decaying signals were examined. Finally, Parseval's Theorem was verified numerically, and the Energy Spectral Density (ESD) was computed to illustrate the distribution of signal energy in the frequency domain.